TP 04 — Gestion des processus sous Linux

CO	Description	Ce TP
CO1	Understand OS fundamentals, Linux command-line environment, and operating system service interfaces	—
CO2	Understand process scheduling, memory management, and file system organization algorithms	✓
CO3	Manage Linux users and permissions, implement shell scripts, and apply text processing tools	—

Contexte

Vous êtes **stagiaire devops junior**. Le serveur Linux de production montre des signes de surcharge : l'équipe vous confie une mission de **diagnostic et d'optimisation**. D'abord, vous devez **inspecter** ce qui tourne — qui consomme le CPU, quelle hiérarchie de processus, dans quels états — puis **piloter** proprement des tâches sans casser le système. Ensuite, vous devez **prouver** que vous comprenez comment un processus en crée d'autres : vous lirez, compilerez et exécuterez un petit programme C fourni, et vérifierez l'arbre des processus. Enfin, vous **optimiserez** la répartition du CPU en jouant sur les priorités. La mission se déroule en trois objectifs. À la fin de chaque objectif, un **script interactif** valide automatiquement votre travail et génère un rapport que vous montrez à l'enseignant.

Tableau de mission

Objectif	Mission	Script	Récompense
0	Préparer le poste de travail	— (PDF)	5 pts
1	Observer les processus	`lab04_ex1.sh`	5 pts
2	Piloter les processus (jobs et signaux)	`lab04_ex2.sh`	5 pts
3	Créer des processus et observer l'ordonnancement	`lab04_ex3.sh`	5 pts

Objectifs pédagogiques

À la fin de ce TP, vous serez capable de :

* Observer les processus d'un système (`ps`, `ps aux`, `top`, `htop`, `pstree`, `pgrep`) et lire leur état (colonne `STAT`)
* Comprendre la relation parent-enfant entre processus et visualiser l'arbre des processus
* Lancer, suspendre, reprendre et terminer proprement des tâches en arrière-plan (signaux)
* Expliquer le rôle de `fork` et `wait` et observer un processus zombie
* Relier la priorité `nice` à l'ordonnancement du CPU

Pré-requis et matériel

* Terminal Bash fonctionnel (WSL2 sur Windows, UTM sur macOS, ou émulateur natif sous Linux)
* Avoir assisté au cours CM04 (processus, états, `fork`/`exec`/`wait`, signaux, ordonnancement)
* Avoir réalisé les TP1, TP2 et TP3
* L'archive `scripts_lab04.zip` téléchargée depuis le site du cours (voir Exercice 0)
* Les programmes `forkdemo.c` et `cputask.c`, fournis complets dans l'archive (objectif 3)

* Un compilateur C (`cc` : `gcc` sous Linux, `clang` sous macOS)

Déroulement et barème

Le TP se joue en **3 objectifs** de 20 minutes chacun. Pour chaque objectif, vous lancez le script correspondant (`./lab04_ex1.sh`, `./lab04_ex2.sh`, `./lab04_ex3.sh`) : il vous guide étape par étape, vérifie vos réponses, vous donne des indices en cas d'erreur, puis génère un **rapport** (`report_lab04_exN.txt`) que vous montrez à l'enseignant.

Ouvrez **deux terminaux** dans le répertoire du TP : le **terminal 1** lance le script, le **terminal 2** exécute le travail. L'**Objectif 3** ajoute un **troisième terminal** d'observation (`ps`, `top`, `kill`).

À la fin de chaque objectif, rédigez sur papier une **synthèse libre manuscrite** (notes personnelles) : ce que vous avez fait, pourquoi ça marche, et le lien avec le cours. L'enseignant évalue votre compréhension sur cette synthèse.

Barème (sur 20) : Exercice 0 « Environnement prêt » (5 points) + trois objectifs notés sur 5 points chacun (total 15). Absence de machine opérationnelle = 0/20, quel que soit le reste.

Annexe — Aide-mémoire des commandes du TP

Ce tableau ne remplace pas le cours. En cas de doute, utilisez `man commande` (section 2 : appel système) ou `commande --help`.

Observer les processus

Commande	Syntaxe	Rôle
`ps`	`ps [options]`	État des processus de **ton terminal** ; `-o` : colonnes choisies
`ps aux`	`ps aux` / `ps -ef`	Tous les processus ; `ps -u $USER` : les tiens
`pgrep`	`pgrep -l [motif]`	PID par nom ; `-l` affiche le nom, `-u` filtre par utilisateur
`pstree`	`pstree -p`	Arbre parent-enfant (repli : `ps -o pid,ppid,comm`)
`top`	`top`	Vue dynamique, tri par %CPU ; `q` pour quitter
`htop`	`htop`	Variante enrichie et colorée de `top`
`$$` / `$PPID`	`echo $$ $PPID`	PID du shell courant et de son parent
`kill -l`	`kill -l`	Liste tous les signaux
`man`	`man commande`	Page de manuel
`man -k`	`man -k mot-clé`	Recherche une commande par mot-clé

`man -k` s'appuie sur la base `whatis` : si elle ne renvoie rien, lancez `sudo mandb` (Linux) ou `sudo makewhatis` (macOS).

`pstree` et `htop` sont absents par défaut sous macOS : installez `psmisc` et `htop`, ou utilisez les replis `ps` et `top`.

Piloter et prioriser les processus

Commande	Syntaxe	Rôle
Arrière-plan	`commande &`	Lance la commande en arrière-plan
`jobs`	`jobs`	Liste les tâches du shell (numéro `%1`, `%2`, ...)
`fg`	`fg %1`	Ramène un job au premier plan
`bg`	`bg %1`	Reprend un job suspendu en arrière-plan
Ctrl+Z	(Ctrl+Z)	Suspend le job en avant-plan (SIGTSTP)
Ctrl+C	(Ctrl+C)	Interrompt le job en avant-plan (SIGINT)
`kill`	`kill %1` ou `kill PID`	Envoie un signal (SIGTERM par défaut)
`kill -STOP`	`kill -STOP PID`	Suspend (état T) ; `kill -CONT PID` reprend
`nice`	`nice -n 19 commande`	Lance avec une priorité `nice` (élevée = priorité basse)
`renice`	`renice 10 -p PID`	Change la priorité d'un processus en cours
`time`	`time commande`	Mesure `real` / `user` / `sys`

La valeur `nice` va de `-20` (priorité max) à `+19` (priorité min), défaut `0` ; un utilisateur ordinaire ne peut qu'augmenter `nice`.

Compilation C et appels système

Commande	Syntaxe	Rôle
`cc`	`cc -Wall -Wextra -o prog prog.c`	Compile le code source C (`gcc` / `clang`)
`./prog`	`./prog`	Exécute un programme du répertoire courant
`fork`	`man 2 fork`	Crée un processus fils (retourne `0` au fils, le PID au parent)
`exec`	`man 2 exec`	Remplace l'image du processus courant
`wait`	`man 2 wait`	Attend la fin d'un processus fils
`_exit`	`man 2 _exit`	Termine le processus immédiatement

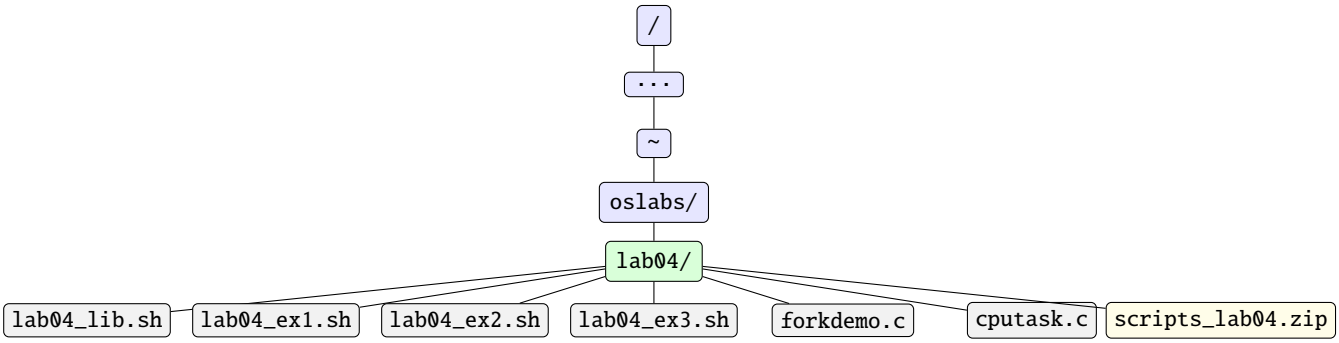
`man 2` = appels système (noyau), `man 3` = fonctions de la bibliothèque C ; `fork`, `exec`, `wait` et `_exit` sont ici des appels système.

1 Exercice 0 — Préparer le poste de travail

Durée : 15 min — 5 points « Environnement prêt »

Contexte

Avant toute mission, on prépare son poste. Ces commandes reviennent à **chaque TP** : à force de répétition, elles deviendront des réflexes.



Instructions

1. Aller à la racine : `cd /`
2. Revenir au répertoire personnel : `cd ~` ou `cd` seul
3. Entrer dans le dossier commun : `cd oslabs`
4. Créer le dossier du TP : `mkdir lab04`
5. Entrer dans le dossier du TP : `cd lab04`
6. Télécharger l'archive :
   `curl -O https://liaoux.github.io/it-intro-to-os/static_files/labs/lab04/scripts_lab04.zip`
7. Extraire les fichiers : `unzip scripts_lab04.zip`
8. Rendre les scripts exécutables : `chmod +x *.sh`
9. Vérifier le compilateur C : `cc --version`
10. Si `cc` est introuvable, l'installer :
    - Linux / WSL : `sudo apt update` puis `sudo apt install gcc`
    - macOS : `xcode-select --install`
11. Installer les outils d'observation (facultatif) : `sudo apt install psmisc htop` (Linux/WSL) ou `brew install psmisc htop` (macOS)
12. Vérifier le contenu du dossier : `ls`

Attendu : 7 fichiers — 4 scripts, 2 programmes C et l'archive `scripts_lab04.zip`.

Appelez le professeur pour valider votre environnement avant de commencer les objectifs.

2 Objectif 1 — Observer les processus

Durée : 20 min — 5 points

Observer les processus du système : les lister, lire leur état, comprendre la relation parent-enfant et surveiller l'activité en temps réel.

Lancez le script (terminal 1) : `./lab04_ex1.sh`

3 Objectif 2 — Piloter les processus (jobs et signaux)

Durée : 20 min — 5 points

Piloter une tâche de bout en bout : la lancer en arrière-plan, la suspendre, la reprendre, puis la terminer proprement à l'aide de signaux.

Lancez le script (terminal 1) : `./lab04_ex2.sh`

4 Objectif 3 — Créer des processus et observer l'ordonnancement

Durée : 20 min — 5 points

Créer des processus en C, observer un zombie, puis expérimenter l'ordonnancement du CPU et les signaux sur une tâche de calcul. Cet objectif nécessite **trois terminaux** : **terminal 1** = script, **terminal 2** = exécution des programmes, **terminal 3** = observation (`ps`, `top`, `kill`).

Lancez le script (terminal 1) : `./lab04_ex3.sh`

Appelez le professeur avec vos rapports `report_lab04_exN.txt` en main, accompagnés d'une synthèse libre manuscrite (notes personnelles) : ce que vous avez fait, pourquoi ça marche, et le lien avec le cours.